

PROJECT PROGRESS REPORT 2

GraphML Studio: Backend Implementation, API Design, Data Flow, Testing & Deployment

GraphML Studio Development Team

April 2026

Executive Summary

This report documents the technical implementation of GraphML Studio's backend systems, including GraphSAGE model architectures, API design, data flow pipelines, testing infrastructure, and cloud deployment. The platform provides a general-purpose browser-based network analysis tool with predictive capabilities using Graph Neural Networks.

1 BACKEND DEVELOPMENT: GRAPHSAGE MODEL IMPLEMENTATIONS

1.1 GraphSAGE Node Classification (GraphSAGE_NC)

Purpose: Predict community membership for nodes in social networks.

Architecture:

- **Layer 1:** SAGEConv(in_channels $\rightarrow$ 128) + BatchNorm + ReLU + Dropout(0.3)
- **Layer 2:** SAGEConv(128 $\rightarrow$ 128) + BatchNorm + ReLU + Dropout(0.3)
- **Layer 3:** SAGEConv(128 $\rightarrow$ num_classes)

Input Features (8-dim): tf, df, tfidf, pmi_max, degree, strength, mean_weight, total_coocc

Training Configuration:

- Loss: Cross-entropy with class weights (inverse frequency)
- Optimizer: Adam (lr=0.005, weight_decay=5e-4)
- Scheduler: ReduceLROnPlateau (factor=0.5, patience=8)
- Split: 5% train, 15% validation, 80% test
- Early stopping: patience=15 epochs on validation Macro-F1
- Max epochs: 100 (full mode), 20 (fast mode)

Embedding Extraction: The `embed()` method returns 128-dimensional node representations from the penultimate layer for t-SNE visualization.

1.2 GraphSAGE Link Prediction (GraphSAGE_LP)

Purpose: Predict likely edges between nodes.

Architecture:

- **Encoder Layer 1:** SAGEConv(in_channels $\rightarrow$ 128) + BatchNorm + ReLU + Dropout(0.3)
- **Encoder Layer 2:** SAGEConv(128 $\rightarrow$ 64)
- **Decoder:** Dot-product: $\text{score}(u,v) = \text{dot}(z_u, z_v)$

Training Configuration:

- Loss: Binary cross-entropy with logits
- Optimizer: Adam (lr=0.001, weight_decay=5e-4)
- Split: RandomLinkSplit (5% train, 15% val, 80% test)
- Negative sampling: ratio=1.0 (1 negative per positive)
- Early stopping: patience=15 epochs on validation AUC-ROC
- Max epochs: 80 (full mode), 20 (fast mode)

Table 1: Model Architecture Comparison

Property	GraphSAGE_NC	GraphSAGE_LP
Purpose	Node classification	Edge prediction
Convolutional layers	3	2
Hidden dimension	128 (full), 64 (fast)	128 (full), 64 (fast)
Output dimension	num_classes (dynamic)	64 (fixed)
Decoder	None	Dot-product
Loss function	Cross-entropy	BCE with logits
Evaluation metric	Accuracy, Macro-F1	AUC-ROC, Avg Precision

1.3 PyTorch Geometric Data Builder

The `build_pyg_data()` function constructs PyG Data objects:

1. **Feature Engineering:** Select available features, apply log-transform to heavy-tailed distributions (tf, df, tfidf, strength, total_cooc)
2. **Standardization:** StandardScaler fit on training data
3. **Label Encoding:** LabelEncoder maps community IDs to contiguous integers
4. **Edge Construction:** Bidirectional expansion for undirected graphs

1.4 Inductive Prediction

For unseen nodes, the system:

1. Builds feature vector from connections to known nodes (structural features only)
2. Extends the graph with new node and edges
3. Runs forward pass on extended graph
4. Extracts prediction for the new node

2 API DESIGN

2.1 Node.js/Express API Endpoints (Port 3000)

Method	Endpoint	Description
POST	/api/analyze	CSV upload, graph construction, community detection, centrality
POST	/api/predict-node	Node classification (toy fallback or GNN proxy)
POST	/api/predict-edge	Link prediction (Adamic-Adar or GNN proxy)
GET	/api/gnn/status	GNN service health check
POST/GET	/api/gnn/wake	GNN cold-start wake mechanism
POST	/api/gnn/demo/*	Demo mode endpoints (proxied)
POST	/api/gnn/train	Train GNN on user graph
GET/POST	/api/gnn/user/*	User graph endpoints (proxied)

2.2 Python/Flask GNN API Endpoints (Port 5001)

Method	Endpoint	Description
GET	/health	Service health check
POST	/gnn/demo/load	Load pre-trained demo models
GET	/gnn/demo/embeddings	Return t-SNE embeddings
GET	/gnn/demo/communities	Return community metadata
POST	/gnn/demo/search	Node search (prefix match)
POST	/gnn/demo/predict-node	Demo node classification
POST	/gnn/train	Train NC + LP on user graph
GET	/gnn/user/embeddings	User graph t-SNE embeddings
POST	/gnn/user/predict-node	User node prediction (inductive)
POST	/gnn/user/predict-edge	User link prediction

2.3 Request/Response Examples

Graph Analysis Request:

```
POST /api/analyze
Content-Type: multipart/form-data
Body: { csv: <File> }
```

Node Prediction Response:

```
{
  "term": "nodeA",
  "predicted_community": 0,
  "theme": "Community Theme",
  "confidence": 78.42,
  "top5": [
    { "community": 0, "probability": 78.42 },
    { "community": 4, "probability": 12.18 },
    ...
  ]
}
```

3 DATA FLOW ARCHITECTURE

3.1 End-to-End Data Flow

3.2 CSV Upload and Parsing Pipeline

1. **Delimiter Detection:** Test candidates [',', ';', '\t', '|'], select most frequent
2. **Header Identification:** Check for hints {source, target, from, to, weight}
3. **Streaming Processing:** csv-parser with auto-detected separator
4. **Edge Validation:** Skip self-loops, empty nodes, duplicates

3.3 Graph Construction Pipeline

1. **Graphology Instantiation:** undirected, non-multi, no self-loops
2. **Edge Deduplication:** First occurrence wins
3. **Degree Maintenance:** Update node degree attribute during edge insertion
4. **Sampling (if needed):** Top-5000 nodes by degree, edge stepping for 15K edges max

3.4 Model Caching System

Graph Hashing: SHA-256 hash of sorted nodes + sorted undirected edges (canonicalized)

cache_id = "{graph_hash}_{mode}_{hidden}_{nc_epochs}_{lp_epochs}"

Example: "59a8ff24939a_fast_64_20_20"

Cache Files:

- cache_{id}_nc.pt — GraphSAGE_NC state dict
- cache_{id}_lp.pt — GraphSAGE_LP state dict
- cache_{id}.json — Metadata (metrics, timestamps)

4 TESTING INFRASTRUCTURE

4.1 Testing Strategy

GraphML Studio employs a dual test suite:

- **Node.js:** Jest + Supertest for Express endpoint testing
- **Python:** pytest + pytest-flask for Flask endpoint and model testing

Test Categorization:

- **Unit tests:** Individual functions in isolation
- **Integration tests:** HTTP endpoints
- **End-to-end tests:** Full pipeline from data to inference

Total Tests: 57 (29 Jest + 28 pytest)

4.2 Node.js Test Suite

Configuration: `jest.config.json`

- Test timeout: 30 seconds
- Module name mappers for ESM packages (networkanalysis-ts)
- Mocks for non-deterministic components (Louvain, Leiden)

Test Coverage:

Category	Tests	Type
<code>detectSeparator()</code>	5	Unit
<code>looksLikeHeaderRow()</code>	4	Unit
<code>POST /api/analyze</code>	11	Integration
<code>POST /api/predict-node</code>	4	Integration
<code>POST /api/predict-edge</code>	5	Integration

4.3 Python Test Suite

Configuration: `conftest.py` with fixtures

Test Coverage:

4.4 Test Execution

Node.js tests:

```
npx jest tests/server.test.js --verbose
```

Python tests:

Category	Tests	Type
TestBuildPygData	6	Unit
TestGraphSAGE_NC	3	Unit
TestGraphSAGE_LP	3	Unit
TestHealthEndpoint	3	Integration
TestDemoEndpoints	5	Integration
TestUserTrainEndpoint	5	Integration
TestTrainingPipeline	2	Unit
TestEndToEnd	1	End-to-End

```
pytest tests/test_gnn_services.py -v
```

```
# With coverage:
```

```
npx jest tests/server.test.js --coverage
```

```
pytest tests/test_gnn_services.py --cov=../gnn_services
```

5 CLOUD DEPLOYMENT ON RENDER.COM

5.1 Deployment Architecture

Two independent web services on Render.com free tier:

- **graphml-studio-web:** Node.js/Express (Port 3000, public URL)
- **graphml-studio-gnn:** Python/Flask (Port 5001, internal)

5.2 Service Configuration (render.yaml)

GNN Python Backend Service:

```
services:
- type: web
  name: graphml-studio-gnn
  runtime: python
  plan: free
  rootDir: graphml-studio
  buildCommand: >-
    pip install --upgrade pip &&
    pip install torch==2.1.0 --index-url https://download.pytorch.org/whl/cpu &&
    pip install pyg_lib torch_scatter torch_sparse -f https://data.pyg.org/whl/torch-2.1.0+cpu &&
    pip install -r gnn_services/requirements.txt
  startCommand: python gnn_services/app.py
  envVars:
  - key: PYTHON_VERSION
    value: "3.11.11"
  - key: PYTHONUNBUFFERED
    value: "1"
```

Node.js Frontend Service:

```

- type: web
  name: graphml-studio-web
  runtime: node
  plan: free
  rootDir: graphml-studio
  buildCommand: npm install
  startCommand: node server.js
  envVars:
    - key: GNN_URL
      value: https://graphml-studio-gnn.onrender.com

```

5.3 Build Pipeline

Python Service (2-3 minutes):

1. Upgrade pip (5s)
2. Install PyTorch CPU-only (60-90s, 180MB)
3. Install PyG dependencies (30-45s)
4. Install application dependencies (15-30s)

Node.js Service (1 minute):

1. npm install (express, graphology, csv-parser, etc.)

5.4 Free Tier Constraints and Mitigation

Constraint	Limit	Mitigation
Memory	512MB per service	CPU-only PyTorch, in-memory processing, graph sampling
Cold start	30-60s after 15min idle	Wake endpoint with polling, 30s health monitoring
Build time	15 minutes max	Pinned versions, cached wheels, staged installation
No internal networking	Public URLs only	Acceptable for low traffic, internal DNS on paid tiers

5.5 Health Monitoring

```

# 30-second polling cycle
function startGNNServiceMonitor() {
  checkGNNService(); // Run immediately
  return setInterval(checkGNNService, 30000); // Then every 30s
}

function checkGNNService() {
  probeGNN().then(({ ok }) => {
    gnnAvailable = ok;
  });
}

```

5.6 Deployment Workflow

1. Developer pushes code to GitHub
2. Render detects push (GitHub integration)
3. Render triggers parallel builds for both services
4. Services start (node server.js, python app.py)
5. Health checks begin
6. New version live

6 CONCLUSION

6.1 Summary of Technical Achievements

1. **Two GraphSAGE Models:** 3-layer NC (54K parameters) and 2-layer LP with dot-product decoder
2. **Comprehensive API:** 14 Express + 10 Flask endpoints covering all analysis tasks
3. **Robust Data Pipeline:** Auto-detecting CSV parser, streaming graph construction, dual community detection, conditional centrality
4. **Inductive Prediction:** Classify unseen nodes from connections
5. **Model Caching:** SHA-256 graph hashing for instant repeated uploads
6. **Comprehensive Testing:** 60 tests (unit, integration, e2e)
7. **Production Deployment:** Two-service architecture with health monitoring and cold-start handling

6.2 Next Development Priorities

- **Model Performance:** Hyperparameter tuning, Graph Attention Networks, ensemble methods
- **API Enhancements:** Graph export (GraphML, GEXF), batch prediction, temporal analysis
- **Deployment:** Switch to gunicorn, add Redis caching, PostgreSQL for persistence
- **Testing:** Performance benchmarks, load testing, 80%+ code coverage

Document prepared by: GraphML Studio Development Team

Report version: 2.0

Date: April 6, 2026

USER (Browser)	Node.js Server (Port 3000)	Python GNN Service (Port 5001)
[1] Upload CSV		
----->	[2] Parse CSV	
	detectSeparator()	
	looksLikeHeaderRow()	
	[3] Build Graph	
	graphology Graph	
	Deduplicate, skip loops	
	[4] Sample if > 5000 nodes	
	Top-N by degree	
	[5] Community Detection	
	Louvain + Leiden	
	[6] Centrality Metrics	
	Betweenness, Eigen, Degree	
	(skip if > 1500 nodes)	
[7] {nodes, edges}		
<-----		
[8] Render vis.js		
[9] Click "Train GNN"		
----->	[10] POST /gnn/train	
	----->	[11] Hash graph
		[12] Cache check
		[13] build_pyg_data
		[14] Train NC (5/15/80)
		[15] Train LP (5/15/80)
		[16] Compute t-SNE
		[17] Save to cache
	[18] {status, metrics}	
	<-----	
[19] Training complete		
<-----		
[20] Classify "term"		
----->	[21] POST /gnn/predict	
	----->	[22] Forward pass
		[23] Softmax
	[24] {community, top5}	
	<-----	
[25] Prediction result		
<-----		